

docs/project-plan.md: "2026-08-24T18:43:57-03:00"

docs/decisions/technical-decisions-phase-03-videos.md: "2026-08-24T18:54:35-03:00"

docs/decisions/technical-decisions-phase-02-auth.md: "2026-05-12T12:23:19-03:00"

docs/decisions/technical-decisions-phase-01-configuracao-base.md: "2026-05-12T12:21:12-03:00"

docs/phases/phase-02-auth/phase-02-auth.md: "2026-08-24T18:43:57-03:00"

phase-03-videos — Context

Scope

Phase name: Fase 03 — Upload e Processamento de Vídeos

Capabilities

- Serviço de armazenamento de arquivos (vídeos e thumbnails)
- Serviço de processamento em segundo plano (filas)
- Upload de vídeos com suporte a arquivos de até 10GB sem impacto na performance
- Pré-cadastro automático do vídeo como rascunho ao iniciar o upload
- Processamento automático do vídeo após upload (extração de duração e metadados)
- Geração automática de thumbnail a partir de um frame do vídeo
- URL única por vídeo, sem conflito com outros vídeos
- Reprodução via streaming (sem necessidade de download completo)
- Download do vídeo pelo usuário

Out of scope: Edição das informações do vídeo (título, descrição, categoria, thumbnail customizada), visibilidade público/unlisted, fluxo de rascunho→publicação, painel de gerenciamento, categorias, página pública do canal, comentários, likes, contagem de visualizações — todos pertencem à Fase 04+ e à Fase 05.

Deliverables: upload de até 10GB funcional (multipart presigned direto para o storage), processamento automático do vídeo (duração + metadados via ffprobe, thumbnail via ffmpeg) em worker isolado, streaming funcionando (presigned GET com Range/206 nativo), URLs únicas geradas (nanoid). Ciclo de status `draft` → `processing` → `ready` | `error` persistido.

Affected subprojects: `nestjs-project/`

Deferred subprojects: `next-frontend/` — a UI de upload, o player de vídeo e o dashboard do canal que consomem estes endpoints ficam diferidos para fases futuras (Fase 04 — Gerenciamento de Vídeos e Canal, Fase 05 — Página de Visualização). Os TDs cross-layer (TD-03 upload, TD-07 streaming) definem o contrato que o frontend consumirá, mas nenhum código de frontend é escrito nesta fase.

Sequencing notes: Depends on Fase 01 — Configuração Base do Projeto e Fase 02 — Cadastro, Login e Gerenciamento de Conta (guards JWT, entidade `Channel`, filtro de exceções de domínio, namespaces de config).

Neighbors (for boundary detection only): Fase 02 — Cadastro, Login e Gerenciamento de Conta (prior), Fase 04 — Gerenciamento de Vídeos e Canal (next).

Decisions Index

Ref	Source	Scope	Topic	Status	Decision	Libraries
phase-03-videos/ TD-01	technical-decisions-phase-03-videos.md	Backend	Message Queue Technology	decided	A (BullMQ + Redis)	@nestjs/bullmq@^11.x, bullmq@^5.x, ioredis@^5.x, redis:7-alpine (Compose)
phase-03-videos/ TD-02	technical-decisions-phase-03-videos.md	Backend	Object Storage Backend & Client SDK	decided	A (MinIO + AWS SDK v3)	@aws-sdk/client-s3@^3.x, @aws-sdk/s3-request-presigner@^3.x, minio (Compose)
phase-03-videos/ TD-03	technical-decisions-phase-03-videos.md	Cross-layer	Large-File (10GB) Upload Strategy	decided	A (Presigned Multipart Upload, direct-to-storage)	— (reuses TD-02 AWS SDK v3)
phase-03-videos/ TD-04	technical-decisions-phase-03-videos.md	Backend	Video Processing Worker Architecture	decided	A (Separate NestJS worker container)	— (reuses @nestjs/bullmq; FFmpeg/ffprobe binaries in worker image)
phase-03-videos/ TD-05	technical-decisions-phase-03-videos.md	Backend	FFmpeg / ffprobe Invocation Approach	decided	B (Direct binary invocation via Node child_process)	— (Node built-in child_process ; fluent-ffmpeg explicitly rejected — archived

Ref	Source	Scope	Topic	Status	Decision	Libraries
						2025-05-22)
phase-03-videos/ TD-06	technical-decisions-phase-03-videos.md	Backend	Unique Video URL Identifier	decided	B (nanoid public id; PK stays uuid)	nanoid@^3.x
phase-03-videos/ TD-07	technical-decisions-phase-03-videos.md	Cross-layer	Video Streaming Strategy	decided	A (Presigned GET URL, native Range/206)	— (reuses TD-02 @aws-sdk/s3-request-presigner)
phase-03-videos/ TD-08	technical-decisions-phase-03-videos.md	Backend	Video Status Life-cycle	decided	A (draft → processing → ready \ error , four-state enum)	— (enum on Video entity; retry via @nestjs/bullmq)

Source files:

- docs/decisions/technical-decisions-phase-03-videos.md

Capability Coverage

Capability	Covered by
Serviço de armazenamento de arquivos (vídeos e thumbnails)	phase-03-videos/TD-02
Serviço de processamento em segundo plano (filas)	phase-03-videos/TD-01, phase-03-videos/TD-04
Upload de vídeos com suporte a arquivos de até 10GB sem impacto na performance	phase-03-videos/TD-03, phase-03-videos/TD-02
Pré-cadastro automático do vídeo como rascunho ao iniciar o upload	phase-03-videos/TD-03, phase-03-videos/TD-08, phase-03-videos/TD-06
Processamento automático do vídeo após upload (extração de duração e metadados)	phase-03-videos/TD-04, phase-03-videos/TD-05, phase-03-videos/TD-01, phase-03-videos/TD-08
Geração automática de thumbnail a partir de um frame do vídeo	phase-03-videos/TD-04, phase-03-videos/TD-05, phase-03-videos/TD-02
URL única por vídeo, sem conflito com outros vídeos	phase-03-videos/TD-06
Reprodução via streaming (sem necessidade de download completo)	phase-03-videos/TD-07, phase-03-videos/TD-02
Download do vídeo pelo usuário	phase-03-videos/TD-07, phase-03-videos/TD-02

Decisions Detail

phase-03-videos/TD-01

Recommendation: Option A (BullMQ + Redis) — For a video platform whose whole point is heavy background processing, BullMQ is the ecosystem standard with the only first-party NestJS module (`@nestjs/bullmq`), giving decorator-based workers, retries/backoff, and a dead-letter set out of the box. The cost is one Redis container, which is trivial to add to Compose and is a common companion service the project will likely reuse (caching, rate-limit store) in later phases.

Libraries: `@nestjs/bullmq@^11.x`, `bullmq@^5.x`, `ioredis@^5.x`, `redis:7-alpine` (Compose service)

phase-03-videos/TD-02

Recommendation: Option A (MinIO + AWS SDK v3) — MinIO gives an S3-compatible object store that runs as one Compose container, matching the diagram and keeping storage self-hosted (consistent with the queue decision). Using the standard AWS SDK v3 (not the MinIO-specific client) means the exact same code path works against real S3 in production by changing only endpoint/credentials — the

best long-term parity. The known MinIO/SDK signing quirks are configuration details (`forcePathStyle: true` , explicit `region / endpoint`), not blockers.

Libraries: `@aws-sdk/client-s3@^3.x` , `@aws-sdk/s3-request-presigner@^3.x` , `minio` (Compose service)

phase-03-videos/TD-03

Recommendation: Option A (Presigned Multipart Upload, direct client → storage) — It is the only option that fully honors both hard requirements: 10GB with zero API data-path impact, and connection-failure resumability (per-part retry). It leverages the S3 multipart API that MinIO already exposes (TD-02), so no new protocol/component is needed. The extra endpoints are a well-understood, bounded cost; part size (~50–100MB) and presigned-URL TTL become explicit policy values. The draft pre-registration (`POST /videos`) is the natural “initiate” step.

Libraries: — (reuses TD-02 `@aws-sdk/client-s3` + `@aws-sdk/s3-request-presigner` — no new dependency)

phase-03-videos/TD-04

Recommendation: Option A (Separate NestJS worker container) — It matches the C4 diagram’s dedicated Video Worker, isolates FFmpeg CPU load from API latency, scales independently, and — crucially — reuses the existing NestJS DI, config (`registerAs`), TypeORM repositories, and storage client with no duplication (one codebase, worker endpoint). The only cost is a second Compose service sharing the same image, which is the standard NestJS pattern for background workers.

Libraries: — (reuses `@nestjs/bullmq` from TD-01; FFmpeg/ffprobe binaries installed in the worker image — invocation approach in TD-05)

phase-03-videos/TD-05

Recommendation: Option B (Direct binary invocation via Node `child_process`) — With `fluent-ffmpeg` dead (archived/unmaintained since 2025-05-22, no support for recent FFmpeg), invoking the binaries directly is the most robust, dependency-free choice; ffprobe’s JSON output and a one-frame ffmpeg command are simple, well-documented invocations. `execa` is a fine ergonomic upgrade, but its ESM-only recent versions add build friction in the CommonJS NestJS project for marginal benefit. Binary paths are pinned via the worker image (`FFMPEG_PATH` / `FFPROBE_PATH` if needed).

Libraries: — (Node built-in `child_process` ; `ffmpeg` + `ffprobe` binaries installed in the worker Docker image). Note: `fluent-ffmpeg` / `@types/fluent-ffmpeg` are explicitly **not** adopted (rejected in TD-05 Option A).

phase-03-videos/TD-06

Recommendation: Option B (nanoid `public_id` , PK stays a UUID) — For a public, YouTube-style URL the identifier must be short, URL-safe, non-enumerable, and collision-safe; nanoid at a safe length (~12–21 chars) delivers all four, while the internal primary key remains a standard PostgreSQL `uuid` (consistent with existing entities). This cleanly separates “internal PK” from “public URL id”. ULID’s sortability helps PKs, not a public lookup column, and its timestamp prefix is an enumeration leak.

Libraries: `nanoid@^3.x` (CommonJS-compatible 3.x line for the NestJS build)

phase-03-videos/TD-07

Recommendation: Option A (Presigned GET URL — direct client ↔ storage streaming) — It is the literal realization of the architecture diagram’s Frontend→Storage “Streams” edge, offloads all byte

transfer and Range/ 206 handling to MinIO/S3 (native), and scales without loading the API — mirroring the direct-to-storage philosophy chosen for uploads (TD-03). Short-lived presigned URLs provide adequate access control for this phase; a 302 -redirect variant can be layered later if a stable API-owned URL is desired. Download reuses the same presigned mechanism with a Content-Disposition: attachment override.

Libraries: — (reuses TD-02 @aws-sdk/s3-request-presigner — no new dependency)

phase-03-videos/TD-08

Recommendation: Option A (draft → processing → ready | error) — It is exactly the lifecycle the plan describes, models cleanly as a single persisted enum column with a small, well-defined transition set, and dovetails with BullMQ's retry/backoff/dead-letter (final failure ⇒ error , retryable back to processing). Public streaming (TD-07) is gated on ready . Orphaned draft uploads are handled by a TTL/cleanup job (aligned with AbortMultipartUpload from TD-03). Granular states are deferred until a management dashboard (Fase 04) justifies them.

Libraries: — (none — enum on the Video entity; retry policy via @nestjs/bullmq from TD-01)

Inherited Decisions Detail

phase-01-configuracao-base/TD-01

Recommendation: Option A (@nestjs/config) — Official, core-team-maintained, guaranteed NestJS 11 compatibility. The registerAs() factory pattern solves the TypeORM CLI sharing problem.

Libraries: @nestjs/config@^4.x

phase-01-configuracao-base/TD-02

Recommendation: Option A (Joi) — First-class integration with @nestjs/config via validationSchema , zero custom wiring, native string-to-number coercion.

Libraries: joi@^17.x

phase-01-configuracao-base/TD-03

Recommendation: Option B (Namespaced/grouped with registerAs) — Clear file boundaries per domain, typed injection via ConfigType<typeof xxxConfig> , natural scalability. The registerAs() factory is dual-purpose: DI token + plain importable function.

Libraries: —

phase-01-configuracao-base/TD-04

Recommendation: Option A (Shared registerAs factory) — data-source.ts imports the factory, calls dotenv.config() , then calls the factory. Zero duplication, minimal code, no extra abstraction.

Libraries: dotenv (transitive via @nestjs/config)

phase-02-auth/TD-06

Recommendation: Option A (class-validator + class-transformer) — The documented NestJS approach; the project already uses decorators extensively. Phase 03 DTOs (upload initiation, complete-upload, update-video) reuse the same global ValidationPipe established here.

Libraries: class-validator@^0.14.x , class-transformer@^0.5.x

phase-02-auth/TD-07

Recommendation: Option A (Custom Domain Exception Filter) — Machine-readable error codes in a `{ statusCode, error, message }` envelope. Phase 03 video-domain errors (`VIDEO_NOT_FOUND` , `NOT_VIDEO_OWNER` , `VIDEO_NOT_READY` , `UPLOAD_NOT_INITIATED` , etc.) extend the same `DomainException` base and are rendered by the inherited `DomainExceptionFilter` .

Libraries: —

phase-02-auth/TD-02 (JWT guard)

Recommendation: Option B as implemented (Custom guards with `@nestjs/jwt` only) — the global `JwtAuthGuard` (`APP_GUARD`) and `@Public()` / `@CurrentUser()` decorators established in Fase 02 are reused verbatim to protect all mutating `/videos` endpoints and to resolve the authenticated owner for ownership checks.

Libraries: `@nestjs/jwt@^11.0.0`

Inherited Conventions

- Backend config uses `@nestjs/config` with namespaced `registerAs(name, () => ({...}))` factories — one file per domain in `src/config/` . Phase 03 adds `storage.config.ts` , `queue.config.ts` , and `upload.config.ts` following this pattern. (from phase 01)
- Env variables are validated by a Joi schema in `src/config/env.validation.ts` , passed to `ConfigModule.forRoot({ validationSchema, validationOptions: { allowUnknown: true, abortEarly: false } })` . All new Phase 03 env vars are added to this schema. (from phase 01)
- Config is injected via `ConfigType<typeof xxxConfig>` and `@Inject(xxxConfig.KEY)` ; the same factory is importable as a plain function for non-DI contexts. (from phase 01)
- `TypeOrmModule.forRootAsync` is used with `autoLoadEntities: true` , `synchronize: false` . Schema changes are made **only** through timestamped migrations generated via `npm run migration:generate` — the `Video` table is created this way. (from phase 01)
- `data-source.ts` registers migrations via the glob `src/database/migrations/*.ts` and entities via `src/**/*.entity.ts` ; the new `video.entity.ts` is discovered automatically. (from phase 01)
- HTTP error responses follow the `{ statusCode: number, error: string, message: string }` envelope; domain errors extend `DomainException` (with `errorCode` + `httpStatus`) and are mapped by the global `DomainExceptionFilter` . Validation errors use `error: 'VALIDATION_ERROR'` . (from phase 02)
- Global `ValidationPipe` runs with `whitelist: true` , `forbidNonWhitelisted: true` , `transform: true` . (from phase 02)
- Authentication is enforced by a global `JwtAuthGuard` (`APP_GUARD`); endpoints opt out with `@Public()` , and the authenticated payload `{ sub, email }` is read via `@CurrentUser()` . (from phase 02)
- Docker Compose services address each other by **service name** (never `localhost`) per the monorepo `CLAUDE.md` Docker Networking rule — the API/worker reach Postgres, Redis, and MinIO via `db` , `redis` , `minio` . (from phase 01/02)
- Test layering follows the `testing-guide-nestjs-project` Skill: unit (`/*.spec.ts`), integration (`/*.integration-spec.ts` , run with `--runInBand`), E2E (`test/*.e2e-spec.ts`). Definition of Done: `tsc --noEmit` clean + lint + full suite green. (from phase 02)

Inherited Deferred Capabilities

- Telas de cadastro/login/confirmação/recuperação (Fase 02) permanecem diferidas — `next-frontend/` não é iniciado nesta fase. Sem impacto no escopo backend da Fase 03.

Non-UI / Deferred Capabilities

Capability	Status	Rationale	TD refs
UI de upload (seleção de arquivo, progresso, chunking client-side)	deferred	<code>next-frontend/</code> não é iniciado nesta fase; o contrato de upload é definido por TD-03 e consumido pelo frontend na Fase 04.	phase-03-videos/TD-03
Player de vídeo (HTML5 <code><video></code> , seek, buffering)	deferred	UI de reprodução pertence à Fase 05; TD-07 define o contrato de streaming (presigned GET) que o player consumirá.	phase-03-videos/TD-07
Painel de gerenciamento de vídeos do canal	deferred	Pertence à Fase 04 (Gerenciamento de Vídeos e Canal).	—
Cleanup job de uploads draft órfãos (TTL + AbortMultiPartUpload)	non-ui	Mencionado em TD-03/TD-08 como mitigação operacional; não é uma capability entregável da Fase 03 (não listada no project-plan). Pode ser adicionado como tarefa de manutenção posterior.	phase-03-videos/TD-03, phase-03-videos/TD-08

Testing Requirements

Refer to the `testing-guide-nestjs-project` Skill for layer requirements per artifact type in `nestjs-project/`. Phase 03 introduces the `Video` entity + migration, the storage client, the BullMQ queue producer + worker processor, the `ffprobe/ffmpeg` command builders, and new HTTP endpoints (upload initiation, part-URL, complete, stream, download, CRUD). Coverage by layer:

- **Unit (`*.spec.ts`):** `public_id` (nanoid) generation, status-transition guards (TD-08 state machine), `ffprobe/ffmpeg` command builders (mocked `child_process`), storage-key derivation, ownership checks, DTO validation.

- **Integration (`*.integration-spec.ts` , `--runInBand`):** Video entity persistence + constraints, the migration runner (table present after `runMigrations` , reverted after `undoLastMigration`), the videos repository, and BullMQ job round-trips (producer enqueues → processor consumes) against a real Redis + MinIO.
- **E2E (`test/*.e2e-spec.ts`):** upload initiation (`POST /videos`), part-URL issuance, `complete-upload` , stream/download presigned redirects, and the video CRUD endpoints — including auth-guard (401), ownership (403), and status-gate (`VIDEO_NOT_READY`) paths.

Specific layer coverage by SI is recorded in `phase-03-videos.md` (per-SI `**Tests:**` tables) and tracked in `progress.md` .